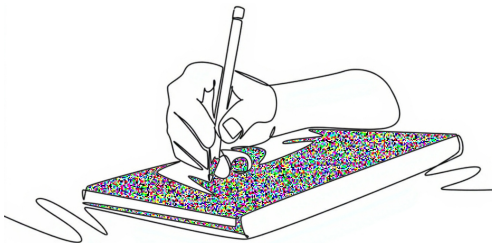


Exploring blind hash-then-sign under FHE

Lorenzo Rovida¹



Topics in Applied Cryptography '25 - Rome, Sept 12th

¹Dept. of Computer Science, University of Milan-Bicocca

Blind signatures

Blind signatures [Cha83] are a form of digital signature where

- ▶ The signer signs a message without knowing its content.
- ▶ The signed message can be unblinded (decrypted) by the user and verified like a normal digital signature.

[Cha83] D.Chaum, *Blind Signatures for Untraceable Payments*, in: CRYPTO '83

Blind signatures

- ▶ Classical applications of blind signatures include e-cash and e-voting.
- ▶ For instance, we could sign encrypted transactions or an encrypted vote, without revealing the content of the ciphertexts.

Blind signatures

- ▶ Classical applications of blind signatures include e-cash and e-voting.
- ▶ For instance, we could sign encrypted transactions or an encrypted vote, without revealing the content of the ciphertexts.
- ▶ **Hash-then-sign** is even more complex to be implemented, as it requires the (blind) evaluation of a hash function.

(Some) related works

- ▶ There are many works that allow to evaluate blind Schnorr [FW24] or ECDSA [DKLS24], but require > 1 rounds and are rather complicated.
- ▶ A recent framework by Lai and Persichetti [LP25] allows concurrently secure code-based blind signature.
- ▶ A recent FHE scheme by Kim and Boneh [BK25] allows to perform fast computations over 256-bits values, but expected ECDSA signature is ≈ 120 mins.

[FW24] G.Fuchsbauer and M.Wolf *Concurrently Secure Blind Schnorr Signatures*, in: EUROCRYPT '23

[DKLS24] J.Doerner, Y.Kondi, E.Lee and A.Shelat *Threshold ECDSA in three rounds*, in: IEEE S&P '23

[LP25] Y.Lai and E.Persichetti *LEAF: Compact and Efficient Blind Signature from Code-based Assumptions*, in: ePrint Archive (2025/1585)

[KB25] J.Kim and D.Boneh *Homomorphic Encryption for Large Integers from Nested Residue Number Systems*, in: CRYPTO '25

Research question(s)

Since FHE is becoming more and more practical, we asked ourselves the following questions.

Research question(s)

Since FHE is becoming more and more practical, we asked ourselves the following questions.

Research question 1

Can we build an efficient framework to allow for practical blind hash-then-sign under FHE?

Research question(s)

Since FHE is becoming more and more practical, we asked ourselves the following questions.

Research question 1

Can we build an efficient framework to allow for practical blind hash-then-sign under FHE?

Research question 2

Can we embed such framework in a *real-world* use case?

The main application

- ▶ As a reference use case, we will deal with blockchain transactions.
- ▶ In particular, we want to extend the framework by Passerat-Palmbach [Pas25] in which a solution for blind arbitrage on decentralized exchanges is proposed.
- ▶ In practice, this solution allows searchers to backrun a user transaction **without seeing the transaction**.

[Pas25] J. Passerat-Palmbach, *Leveraging Homomorphic Encryption for Maximally Extractable Value (MEV) Mitigation: Enabling Blind Arbitrage on Decentralised Exchanges*, in: Financial Cryptography '25

The main application

- ▶ As a reference use case, we will deal with blockchain transactions.
- ▶ In particular, we want to extend the framework by Passerat-Palmbach [Pas25] in which a solution for blind arbitrage on decentralized exchanges is proposed.
- ▶ In practice, this solution allows searchers to backrun a user transaction **without seeing the transaction**.

Extension

After the verification, how to sign the message?

[Pas25] J. Passerat-Palmbach, *Leveraging Homomorphic Encryption for Maximally Extractable Value (MEV) Mitigation: Enabling Blind Arbitrage on Decentralised Exchanges*, in: Financial Cryptography '25

The main application

- ▶ As a reference use case, we will deal with blockchain transactions
- ▶ In particular, we will focus on the use case of **Passerats arbitrage**
- ▶ In practice, we will deal with **transaction verification**

WHAT WE HAVE?



A VERIFIED TRANSACTION!



WHAT WE WANT?



A SIGNED TRANSACTION!



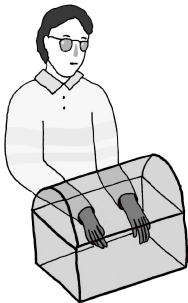
Extension

After the verification

[Pas25] J. Passerat-Palmbach, *Leveraging Homomorphic Encryption for Maximally Extractable Value (MEV) Mitigation: Enabling Blind Arbitrage on Decentralised Exchanges*, in: Financial Cryptography '25

The abstract framework

The (untrusted) signer



$\xleftarrow{\text{enc}(m)_{\text{FHE}}, \text{enc}(sk_{\text{sig}})_{\text{FHE}}}$

$\xrightarrow{\text{enc}(\text{sign}(\text{hash}(m)))_{\text{FHE}}}$

The user



The ingredients

- ▶ The required ingredients for our framework are: a **fully homomorphic encryption** (FHE) scheme, a **hash function** and a **signature scheme**.

The ingredients

- ▶ The required ingredients for our framework are: a **fully homomorphic encryption** (FHE) scheme, a **hash function** and a **signature scheme**.
- ▶ Issue 1: computations in FHE schemes are expensive, and hashing functions require a large computational depth (as circuits).

The ingredients

- ▶ The required ingredients for our framework are: a **fully homomorphic encryption** (FHE) scheme, a **hash function** and a **signature scheme**.
- ▶ Issue 1: computations in FHE schemes are expensive, and hashing functions require a large computational depth (as circuits).
- ▶ Issue 2: domains are usually not equivalent (FHE works with either small integers or approximate reals).

The ingredients

- ▶ It turns out that the recent GBFV [GV25b] scheme can naturally fit the Poseidon2 hash function [GKS23], as they both work in a Goldilocks field!

[GV25b] R. Geelen and F. Vercauteren, *Fully Homomorphic Encryption for Cyclotomic Prime Moduli*, in: EUROCRYPT '25

[GKS23] L. Grassi, D. Khovratovich and M. Schofnegger, *Poseidon2: A Faster Version of the Poseidon Hash Function*, in: AFRICACRYPT '23

The ingredients

- ▶ It turns out that the recent GBFV [GV25b] scheme can naturally fit the Poseidon2 hash function [GKS23], as they both work in a Goldilocks field!
- ▶ Idea → evaluate Poseidon2 under GBFV.

[GV25b] R. Geelen and F. Vercauteren, *Fully Homomorphic Encryption for Cyclotomic Prime Moduli*, in: EUROCRYPT '25

[GKS23] L. Grassi, D. Khovratovich and M. Schofnegger, *Poseidon2: A Faster Version of the Poseidon Hash Function*, in: AFRICACRYPT '23

The ingredients

Poseidon2 under GBFV

- ▶ GBFV support encryptions of vectors from the Goldilocks field (i.e., mod $\Phi_6(2^{32}) = 2^{64} - 2^{32} + 1$)
- ▶ There are already applications that instantiate GBFV over Goldilocks prime [GBK⁺24].
- ▶ Poseidon2 is a permutation function that operates in $\mathbb{F}_{\Phi_6(2^{32})}$, so the two go nicely together in a natural manner.
- ▶ Can be simply turned into a hash function via sponge construction.

[GBK⁺24] M.Gama, E.Beni, J.Kang, J.Spiessens and F.Vercauteren, *Blind zkSNARKs for Private Proof Delegation and Verifiable Computation over Encrypted Data*, in: ePrint Archive 2024/1684

The ingredients - RQ1

Poseidon2 under GBFV

- ▶ Under the POSEIDON2_GOLDDILOCKS_8_PARAMS set of parameters, Poseidon2 can permute 8 elements from $\mathbb{F}_{\Phi_6(2^{32})}$.
- ▶ Roughly 60 bytes of data¹².
- ▶ The circuit is somewhat shallow

¹²Assuming each element from the field is represented by 63 bits

The ingredients

Poseidon 2 under GBFV

Algorithm 1 The Poseidon2 permutation function, under Goldilocks 8 parameters

```
1: state  $\leftarrow$  matmul_external( $m$ )
2: for  $i \leftarrow 0$  to 3 do
3:   state  $\leftarrow$  add_rc(state, RC8[ $i$ ])
4:   state  $\leftarrow$  sbox(state)
5:   state  $\leftarrow$  matmul_external(state)
6: end for
7: for  $i \leftarrow 0$  to 21 do
8:   state[0]  $\leftarrow$  state[0] + RC8[ $i + 4$ ][0]
9:   state[0]  $\leftarrow$  sbox(state[0])
10:  state  $\leftarrow$  matmul_internal(state, MAT_DIAG8_M_1)
11: end for
12: for  $i \leftarrow 0$  to 3 do
13:  state  $\leftarrow$  add_rc(state, RC8[22 + 4 +  $i$ ])
14:  state  $\leftarrow$  sbox(state)
15:  state  $\leftarrow$  matmul_external(state)
16: end for
```

The ingredients

Poseidon 2 under GBFV

Algorithm 2 The Poseidon2 permutation function, under Goldilocks 8 parameters

```
1: state  $\leftarrow$  matmul_external( $m$ )
2: for  $i \leftarrow 0$  to 3 do
3:   state  $\leftarrow$  add_rc(state, RC8[ $i$ ]) 
4:   state  $\leftarrow$  sbox(state)
5:   state  $\leftarrow$  matmul_external(state)
6: end for
7: for  $i \leftarrow 0$  to 21 do
8:   state[0]  $\leftarrow$  state[0] + RC8[ $i + 4$ ][0]
9:   state[0]  $\leftarrow$  sbox(state[0])
10:  state  $\leftarrow$  matmul_internal(state, MAT_DIAG8_M_1)
11: end for
12: for  $i \leftarrow 0$  to 3 do
13:  state  $\leftarrow$  add_rc(state, RC8[22 + 4 +  $i$ ])
14:  state  $\leftarrow$  sbox(state)
15:  state  $\leftarrow$  matmul_external(state)
16: end for
```

Addition by constant

The ingredients

Poseidon 2 under GBFV

Algorithm 3 The Poseidon2 permutation function, under Goldilocks 8 parameters

```
1: state  $\leftarrow$  matmul_external( $m$ )
2: for  $i \leftarrow 0$  to 3 do
3:   state  $\leftarrow$  add_rc(state, RC8[ $i$ ])
4:   state  $\leftarrow$  sbox(state)
5:   state  $\leftarrow$  matmul_external(state)
6: end for
7: for  $i \leftarrow 0$  to 21 do
8:   state[0]  $\leftarrow$  state[0] + RC8[ $i + 4$ ][0]
9:   state[0]  $\leftarrow$  sbox(state[0])
10:  state  $\leftarrow$  matmul_internal(state, MAT_DIAG8_M_1)
11: end for
12: for  $i \leftarrow 0$  to 3 do
13:  state  $\leftarrow$  add_rc(state, RC8[22 + 4 +  $i$ ])
14:  state  $\leftarrow$  sbox(state)
15:  state  $\leftarrow$  matmul_external(state)
16: end for
```

S-box ($x \mapsto x^7$)



The ingredients

Poseidon 2 under GBFV

Algorithm 4 The Poseidon2 permutation function, under Goldilocks 8 parameters

```
1: state  $\leftarrow$  matmul_external( $m$ )
2: for  $i \leftarrow 0$  to 3 do
3:   state  $\leftarrow$  add_rc(state, RC8[ $i$ ])
4:   state  $\leftarrow$  sbox(state)
5:   state  $\leftarrow$  matmul_external(state)
6: end for
7: for  $i \leftarrow 0$  to 21 do
8:   state[0]  $\leftarrow$  state[0] + RC8[ $i + 4$ ][0]
9:   state[0]  $\leftarrow$  sbox(state[0])
10:  state  $\leftarrow$  matmul_internal(state, MAT_DIAG8_M_1)
11: end for
12: for  $i \leftarrow 0$  to 3 do
13:  state  $\leftarrow$  add_rc(state, RC8[22 + 4 +  $i$ ])
14:  state  $\leftarrow$  sbox(state)
15:  state  $\leftarrow$  matmul_external(state)
16: end for
```

Matrix multiplication



The ingredients

Poseidon 2 under GBFV

Algorithm 5 The Poseidon2 permutation function, under Goldilocks 8 parameters

```
1: state  $\leftarrow$  matmul_external( $m$ )
2: for  $i \leftarrow 0$  to 3 do
3:   state  $\leftarrow$  add_rc(state, RC8[ $i$ ])
4:   state  $\leftarrow$  sbox(state)
5:   state  $\leftarrow$  matmul_external(state)
6: end for
7: for  $i \leftarrow 0$  to 21 do
8:   state[0]  $\leftarrow$  state[0] + RC8[ $i + 4$ ][0] 
9:   state[0]  $\leftarrow$  sbox(state[0])
10:  state  $\leftarrow$  matmul_internal(state, MAT_DIAG8_M_1)
11: end for
12: for  $i \leftarrow 0$  to 3 do
13:  state  $\leftarrow$  add_rc(state, RC8[22 + 4 +  $i$ ])
14:  state  $\leftarrow$  sbox(state)
15:  state  $\leftarrow$  matmul_external(state)
16: end for
```

First slot addition

The ingredients

Poseidon 2 under GBFV

Algorithm 6 The Poseidon2 permutation function, under Goldilocks 8 parameters

```
1: state  $\leftarrow$  matmul_external( $m$ )
2: for  $i \leftarrow 0$  to 3 do
3:   state  $\leftarrow$  add_rc(state, RC8[ $i$ ])
4:   state  $\leftarrow$  sbox(state)
5:   state  $\leftarrow$  matmul_external(state)
6: end for
7: for  $i \leftarrow 0$  to 21 do
8:   state[0]  $\leftarrow$  state[0] + RC8[ $i + 4$ ][0]
9:   state[0]  $\leftarrow$  sbox(state[0])
10:  state  $\leftarrow$  matmul_internal(state, MAT_DIAG8_M_1)
11: end for
12: for  $i \leftarrow 0$  to 3 do
13:   state  $\leftarrow$  add_rc(state, RC8[22 + 4 +  $i$ ])
14:   state  $\leftarrow$  sbox(state)
15:   state  $\leftarrow$  matmul_external(state)
16: end for
```

Matrix multiplication

The ingredients

Poseidon 2 under GBFV

The entire arithmetic circuit requires:

- ▶ `matmul_external`: $3 + 3 = 6$ mult. levels
- ▶ `matmul_internal`: 21 mult. levels
- ▶ `sbox`: $3 \cdot 30$ mult. levels¹³

So in total, 117 multiplicative levels.

¹³Evaluating x^7 with a tree approach requires $\lceil \log_2(7) \rceil = 3$ levels

The ingredients

Poseidon 2 under GBFV

- ▶ Good news 1: because of SIMD operations, we can permute 8 elements (60 bytes) in a single evaluation.

The ingredients

Poseidon 2 under GBFV

- ▶ Good news 1: because of SIMD operations, we can permute 8 elements (60 bytes) in a single evaluation.
- ▶ Good news 2: the depth is somewhat short!

The ingredients

Poseidon 2 under GBFV

- ▶ Good news 1: because of SIMD operations, we can permute 8 elements (60 bytes) in a single evaluation.
- ▶ Good news 2: the depth is somewhat short!
- ▶ Bad news: the evaluation of the circuit might require bootstrapping.

The ingredients

Poseidon 2 under GBFV

- ▶ Good news: because of SIMD operations, we can permute 8 elements (60 bytes) in a single evaluation.
- ▶ Good news 2: the depth is somewhat short!
- ▶ Bad news: the evaluation of the circuit might require bootstrapping.
- ▶ Good news for bad news: bootstrapping for GBFV is getting more and more practical [GV25a, Kim25].

[GV25a] R.Geelen and F.Vercauteren, *Better GBFV Bootstrapping and Faster Encrypted Edit Distance Computation*, in: ePrint Archive 2025/1104

[Kim25] J. Kim, *Bootstrapping GBFV with CKKS*, in: ePrint Archive 2025/888

The ingredients

HAWK

- ▶ HAWK [DPPW22] is a post-quantum signature scheme based on the hardness of module-LIP, a structured version of the Lattice isomorphism problem.
- ▶ Currently the only lattice candidate for the NIST's additional call for post-quantum signatures.
- ▶ Signatures are very simple, can be evaluated in just a few steps – it well suits FHE or MPC implementations.

[DPPW22] L.Ducas, E.Postlethwaite, L.Pulles, W. van Woerden, *HAWK: Module LIP Makes Lattice Signatures Fast, Compact and Simple*, in: ASIACRYPT '22

The ingredients

HAWK

- ▶ The keygen of HAWK is run by the user, which then gives the signer an encrypted version of the sk .

The ingredients

HAWK

- ▶ The keygen of HAWK is run by the user, which then gives the signer an encrypted version of the sk .
- ▶ A signature can be interpreted geometrically (simplified):

The ingredients

HAWK

- ▶ The keygen of HAWK is run by the user, which then gives the signer an encrypted version of the sk .
- ▶ A signature can be interpreted geometrically (simplified):
 1. Given some hash $\mathbf{h} \in \{0, 1\}^{2n}$, find the lattice point $\mathbf{t} \leftarrow B \cdot \frac{1}{2}\mathbf{h} \in \mathcal{L}(B)$ with B being some good lattice basis

The ingredients

HAWK

- ▶ The keygen of HAWK is run by the user, which then gives the signer an encrypted version of the sk .
- ▶ A signature can be interpreted geometrically (simplified):
 1. Given some hash $\mathbf{h} \in \{0, 1\}^{2n}$, find the lattice point $\mathbf{t} \leftarrow B \cdot \frac{1}{2}\mathbf{h} \in \mathcal{L}(B)$ with B being some good lattice basis
 2. Sample some coset

$$\mathbf{x} \leftarrow \mathcal{D}_{\mathbb{Z}^{2n} + \frac{1}{2}\mathbf{t}, \sigma_{\text{sign}}}$$

The ingredients

HAWK

- ▶ The keygen of HAWK is run by the user, which then gives the signer an encrypted version of the sk .
- ▶ A signature can be interpreted geometrically (simplified):
 1. Given some hash $\mathbf{h} \in \{0, 1\}^{2n}$, find the lattice point $\mathbf{t} \leftarrow B \cdot \frac{1}{2}\mathbf{h} \in \mathcal{L}(B)$ with B being some good lattice basis
 2. Sample some coset

$$\mathbf{x} \leftarrow \mathcal{D}_{\mathbb{Z}^{2n} + \frac{1}{2}\mathbf{t}, \sigma_{\text{sign}}}$$

3. A signature becomes the tuple

$$\left(\mathbf{h}, \frac{1}{2}\mathbf{h} - B^{-1} \cdot \mathbf{x} \right)$$

The ingredients

HAWK

- ▶ The keygen of HAWK is run by the user, which then gives the signer an encrypted version of the sk .
- ▶ A signature can be interpreted geometrically (simplified):
 1. Given some hash $\mathbf{h} \in \{0, 1\}^{2n}$, find the lattice point $\mathbf{t} \leftarrow B \cdot \frac{1}{2}\mathbf{h} \in \mathcal{L}(B)$ with B being some good lattice basis
 2. Sample some coset

$$\mathbf{x} \leftarrow \mathcal{D}_{\mathbb{Z}^{2n} + \frac{1}{2}\mathbf{t}, \sigma_{\text{sign}}}$$

3. A signature becomes the tuple

$$\left(\mathbf{h}, \frac{1}{2}\mathbf{h} - B^{-1} \cdot \mathbf{x} \right)$$

4. To check if the signature is valid, check the distance between the two hash and the syndrome.

The ingredients

HAWK

- ▶ Mostly linear operations, apart from the discrete Gaussian sampler $\mathcal{D}$.
- ▶ With precomputed resources and a source of uniformly distributed bits, we can sample in constant time independently from the input.
- ▶ Can we do that also under FHE?
- ▶ Reference: Algorithm 14 [BBD⁺25]

Conclusion

Open questions and directions

- ▶ FHE allows to evaluate many functions, from machine learning to private information retrieval
- ▶ For blind signatures, we are still lacking a practical implementation
- ▶ Can GBFV+Poseidon2+HAWK represent a possible solution?

Conclusion

Thank you! Questions? Suggestions? Critics? Ideas? :-)

When I submit an abstract to a conference without finishing the full paper.



References I



Joppe W. Bos, Olivier Bronchain, Leo Ducas, Serge Fehr, Yu-Hsuan Huang, Thomas Pornin, Eamonn W. Postlethwaite, Thomas Prest, Ludo N. Pulles, and Wessel van Woerden.
HAWK.

Tech. rep., National Institute of Standards and Technology, 2025.



Dan Boneh and Jaehyung Kim.

Homomorphic encryption for large integers from nested residue number systems.

Cryptology ePrint Archive, Paper 2025/346, 2025.



David Chaum.

Blind signatures for untraceable payments.

In *Advances in Cryptology*, CRYPTO '83, pages 199–203, 1983.

References II



Jack Doerner, Yashvanth Kondi, Eysa Lee, and Abhi Shelat.
Threshold ECDSA in Three Rounds .

In *2024 IEEE Symposium on Security and Privacy (S&P)*,
2024.



Léo Ducas, Eamonn W. Postlethwaite, Ludo N. Pulles, and
Wessel van Woerden.

Hawk: Module lip makes lattice signatures fast, compact and
simple.

In *Advances in Cryptology – ASIACRYPT 2022*, pages 65–94,
2022.



Georg Fuchsbauer and Mathias Wolf.

Concurrently secure blind schnorr signatures.

In *Advances in Cryptology – EUROCRYPT 2024*, pages
124–160, 2024.

References III



Mariana Gama, Emad Heydari Beni, Jiayi Kang, Jannik Spiessens, and Frederik Vercauteren.

Blind zkSNARKs for private proof delegation and verifiable computation over encrypted data.

Cryptography ePrint Archive, Paper 2024/1684, 2024.



Lorenzo Grassi, Dmitry Khovratovich, and Markus Schafneggger.

Poseidon2: A faster version of the poseidon hash function.

In *Progress in Cryptology - AFRICACRYPT 2023: 14th International Conference on Cryptology in Africa, Sousse, Tunisia, July 19-21, 2023, Proceedings*, page 177203, 2023.



Robin Geelen and Frederik Vercauteren.

Better GBFV bootstrapping and faster encrypted edit distance computation.

Cryptography ePrint Archive, Paper 2025/1104, 2025.

References IV



Robin Geelen and Frederik Vercauteren.

Fully homomorphic encryption for cyclotomic prime moduli.
2025.



Jaehyung Kim.

Bootstrapping GBFV with CKKS.
Cryptography ePrint Archive, Paper 2025/888, 2025.



Yi-Fu Lai and Edoardo Persichetti.

LEAF: Compact and efficient blind signature from code-based assumptions.
Cryptography ePrint Archive, Paper 2025/1585, 2025.



Jonathan Passerat-Palmbach.

Leveraging homomorphic encryption for maximally extractable value (mev) mitigation: Enabling blind arbitrage on decentralised exchanges.

In *Preproceedings of Financial Cryptography '25, FC '25*, 2025.

References V